RateMyResume

Technical Documentation

A beginner-friendly guide to the codebase: architecture, setup, running, debugging, and changing the project.

Based on a read-through of the repository (backend, frontend, infra, tests). Items the repository does not confirm are tagged **[Not verified]**. No secrets appear in this document; all values are placeholders.

Contents

1. Project overview 3
2. Architecture and diagrams 3
3. Folder structure explained 7
4. Development environment setup (macOS) 8
5. Environment variables and configuration 8
6. Starting, stopping, restarting 10
7. Frontend explained 10
8. Backend and API explained 11
9. Database explained 11
10. Resume ingestion, extraction, OCR, and AI analysis 12
11. Authentication, authorization, and security 12
12. Background jobs and reliability 13
13. Testing and debugging 13
14. Making changes safely 14
15. Deployment and production considerations 14
16. Glossary and quick reference 14

1. Project overview

RateMyResume lets a signed-in user upload resumes (**PDF or DOCX** only; scanned PDFs are supported through OCR). The system extracts the text, identifies candidate details (name, email, phone, location, skills), gives a 0-100 completeness score using fixed rules, and writes an AI-generated review with strengths and improvements. All AI runs locally through Ollama. Admins can create and list users.

User journey: sign in (Keycloak) - upload on /upload - job queued - background worker extracts text - analysis runs automatically - user views status on /resumes and the score, summary, and rubric on /resumes/{id}. The dashboard shows aggregate stats.

Implemented: everything above, retries, caching, rate limiting, duplicate detection, bulk upload (max 20 files). **Not present:** Docker/compose files, README at the repo root, CI configuration, frontend tests, an exported Keycloak realm.

1.1 Technology stack

Technology	Purpose	Why it is used	Where
Next.js 16 + React 19 + TypeScript	Web UI	File-based pages, strong typing	frontend/src/app
Tailwind CSS 4	Styling	Utility classes, no separate CSS files	frontend/src, globals.css
keycloak-js	Browser login	Standard OIDC login and token refresh	frontend/src/lib/keycloak.ts
FastAPI	Backend API	Fast, auto-validated Python web framework	backend/app/main.py
PostgreSQL + SQLAlchemy + Alembic	Database, ORM, migrations	Reliable relational storage; schema history	backend/app/models, alembic
Celery + Redis	Background jobs	Slow work must not block uploads	backend/app/tasks
Redis (db1)	Cache and rate limits	Fast repeated reads; abuse control	backend/app/core
Keycloak	Identity provider	Handles passwords, roles, tokens safely	infra/keycloak
Ollama (llama3.2:3b)	Local LLM	Private: resumes never leave your machine	backend/app/ai/llm.py
PaddleOCR	OCR	Reads text from scanned PDFs	backend/app/ai/ocr.py

2. Architecture and diagrams

The diagrams below show the real components. **Not implemented:** object storage (files are saved to a local folder), a vector database, embeddings.

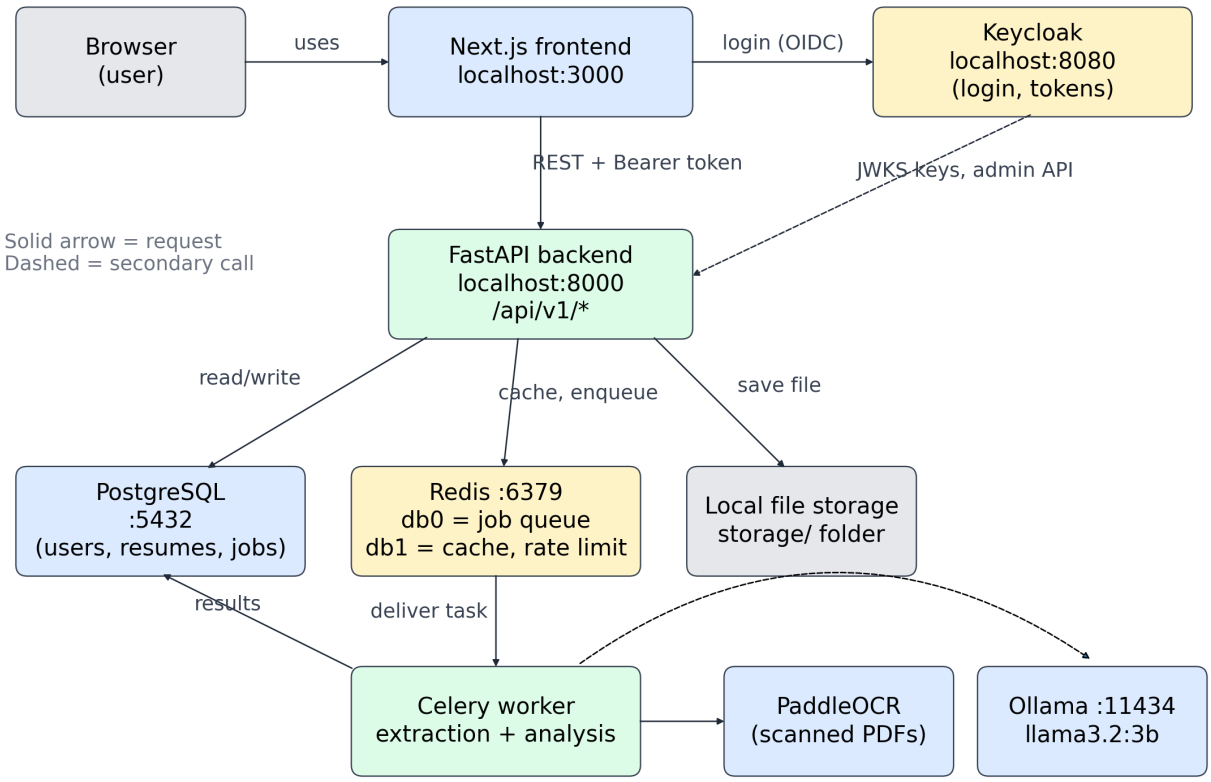


Figure 1. System architecture.

In plain English: the browser loads the Next.js site, which sends the user to Keycloak to log in. Keycloak hands back a token (a signed proof of identity). The frontend attaches it to every call to the FastAPI backend. The backend reads and writes PostgreSQL and the storage folder, and puts slow work on a Redis queue. A Celery worker picks it up and uses OCR and the local LLM.

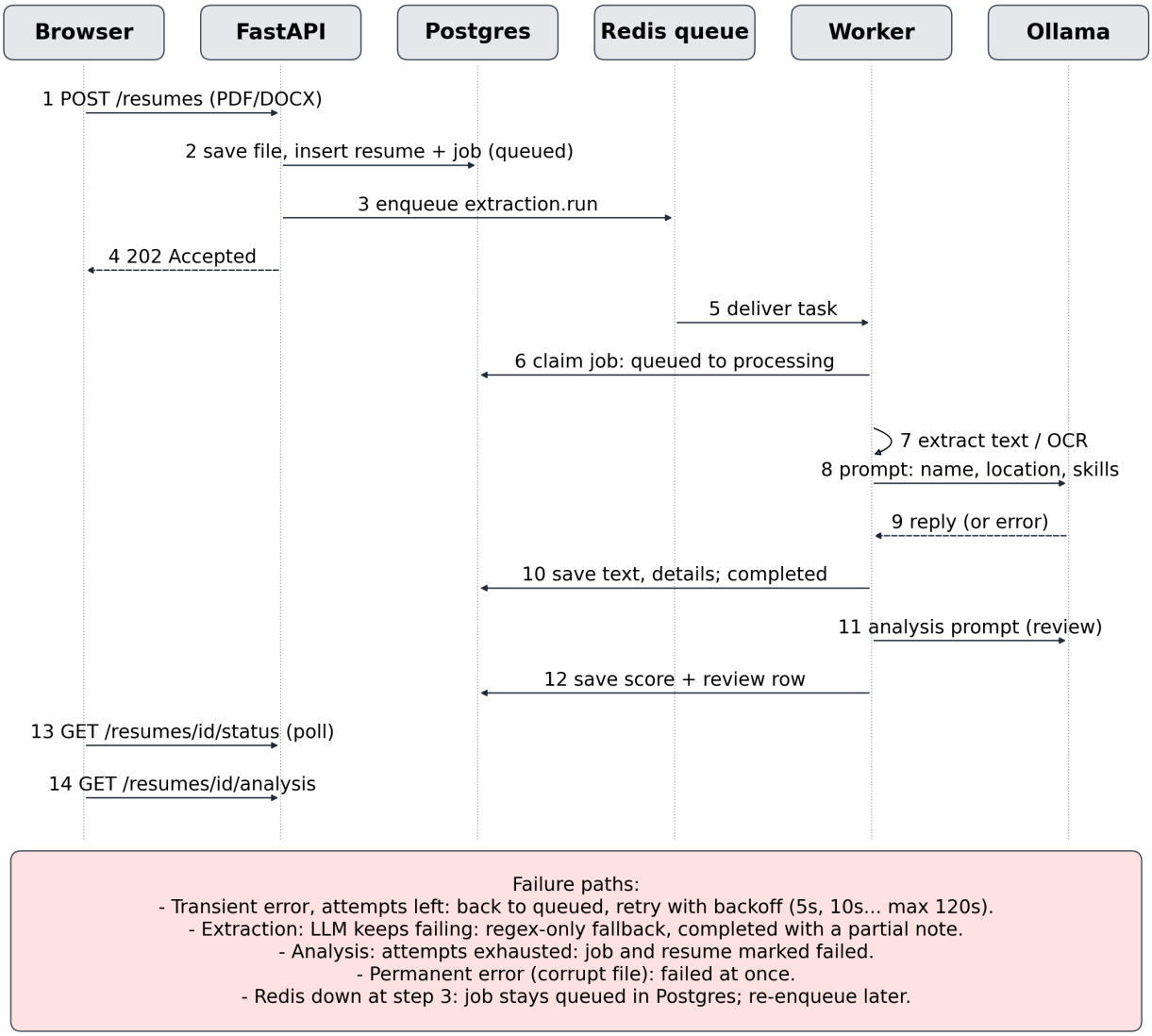


Figure 2. Upload-to-results sequence, with failure paths.

In plain English: uploading returns immediately (HTTP 202, meaning 'accepted, not finished'). The worker does the slow steps afterwards, and the page polls the status endpoint. **Asynchronous** means the work happens later, outside the request.

Request flow and startup: `app/main.py` creates the FastAPI app, adds middleware (body-size limit, security headers, CORS), and mounts five routers under `/api/v1`. A request passes middleware, then the route's auth dependency (JWT check, role check), then a service function, then the database session from `db/session.py`. The frontend's `AuthProvider` forces login on load, and `apiFetch` adds the token.

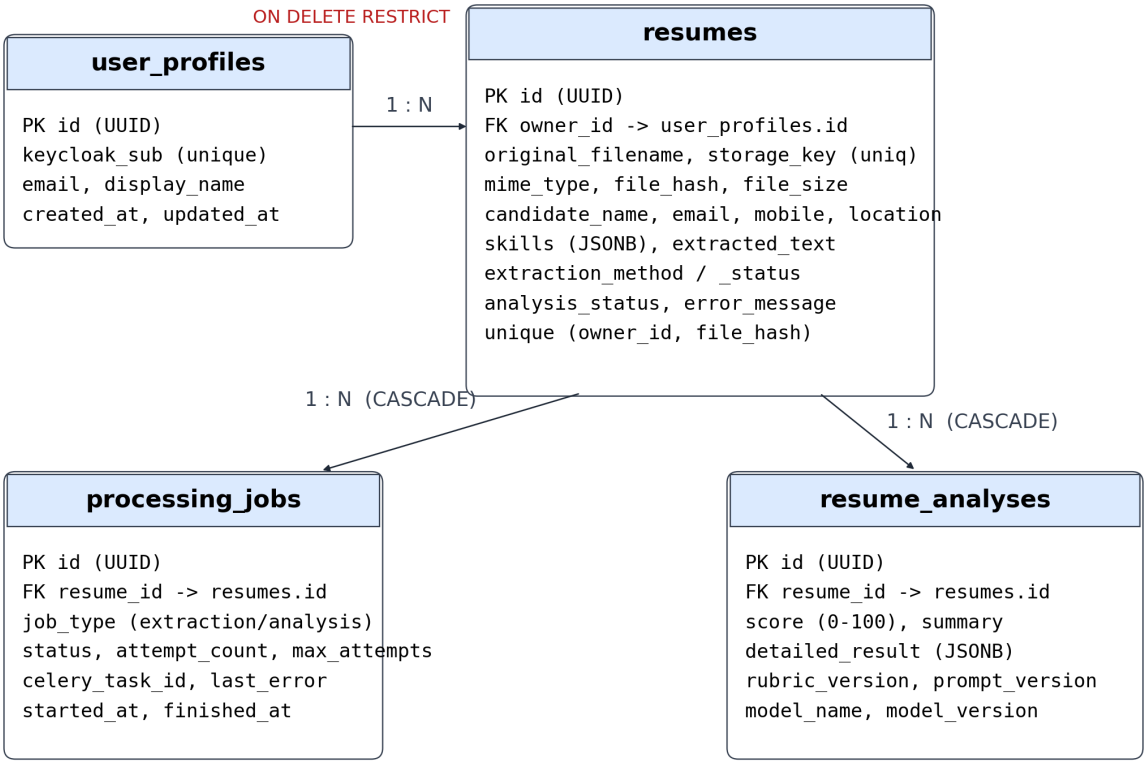
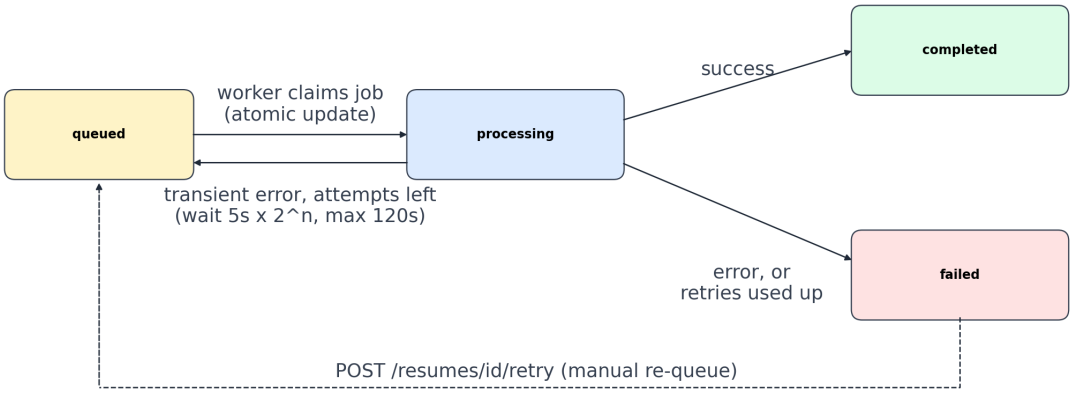


Figure 3. Database entity-relationship diagram.

In plain English: one user owns many resumes. Each resume has many jobs and many analysis results. Deleting a resume deletes its jobs and analyses; a user with resumes cannot be deleted. PK = primary key (unique row id), FK = foreign key (a link to another table's row).



Enqueue failure (Redis down): the job simply stays queued in Postgres; helper scripts in backend/scripts re-enqueue queued jobs.
On success, an extraction job automatically creates and enqueues an analysis job.

Figure 4. Background-job lifecycle.

In plain English: a job waits as *queued*, becomes *processing* when a worker claims it, and ends *completed* or *failed*. Temporary problems send it back to queued with a growing delay. The attempt counter lives in Postgres, so it survives worker restarts.

3. Folder structure explained

```
RateMyResume/
  .gitignore
  infra/keycloak/start.sh        # starts Keycloak in dev mode
  backend/
    requirements.txt  alembic.ini  smoke_test_ocr.py
    alembic/versions/            # one migration: core tables
    app/
      main.py                    # app entry, middleware, /health
      api/v1/                    # routers: me admin resume analysis dashboard
      schemas/                   # Pydantic request/response shapes
      services/                  # business logic
      models/  db/               # tables, engine/session
      auth/security.py           # JWT validation, roles
      core/                      # config, cache, rate_limit, middleware
      ai/                        # llm, ocr, candidate_extraction, analysis
      tasks/                     # celery_app, extraction/analysis tasks, dispatch
    scripts/                     # verify_*.py, requeue helpers, try_*.py
    tests/                       # test_rubric, extraction, ocr_fallback, cache, security...
  frontend/
    package.json  next.config.ts  eslint.config.mjs  tsconfig.json
    src/app/                     # pages: / dashboard upload resumes resumes/[id] admin/users
    src/components/              # NavBar AuthProvider ScoreRing StatusBadge ui
    src/lib/                     # api.ts keycloak.ts resumes.ts dashboard.ts admin.ts
    src/types/api.ts             # TypeScript types mirroring API
```

Path	Purpose / when you edit it	Depends on
backend/app/main.py	Creates the app, middleware, CORS, /health. Edit to add a router or middleware.	api/v1, core
backend/app/core/config.py	All settings, read from the root .env. Edit to add a setting.	everything
backend/app/api/v1/resume.py	Upload, list, detail, status, retry, delete. Thin: calls services.	services, schemas, auth
backend/app/services/resume_service.py	Upload ingestion: type check (magic bytes), size, hash dedupe, store file, create job.	storage, models
backend/app/services/processing.py	Extraction job logic, status transitions, retries.	ai, extraction
backend/app/services/rubric.py	Deterministic 0-100 scoring. Edit to change scoring rules.	none
backend/app/ai/analysis.py	LLM review prompt and output cleanup. Edit to change the prompt.	llm.py
backend/app/auth/security.py	Validates Keycloak JWTs; require_role().	config
backend/app/tasks/*.py	Celery config and tasks; thin wrappers over services.	services
backend/alembic/versions/	Migration history. Add a new file per schema change.	models
frontend/src/lib/api.ts	apiFetch: refreshes token, adds Authorization header, parses errors.	keycloak.ts
frontend/src/components/AuthProvider.tsx	Initializes Keycloak (login-required) and exposes useAuth().	keycloak-js
frontend/src/app/upload/page.tsx	Upload UI (calls lib/resumes.ts).	lib, components
infra/keycloak/start.sh	Loads root .env and starts Keycloak with Postgres.	KEYCLOAK_HOME

Layer boundaries: routers handle HTTP only; services hold logic and are reused by both routers and Celery tasks; models define tables; ai/ wraps the LLM and OCR behind small interfaces. The layering is clean and consistent across the backend. The frontend keeps API calls in `src/lib` and UI in `src/app` and `components`.

4. Development environment setup (macOS)

The repository has no setup guide, Docker files, or Keycloak realm export, so the steps below are derived from the code and `start.sh`. Versions are recommendations **[Not verified]**.

4.1 Install services

```
brew install python@3.12 node postgresql@16 redis ollama      # recommended versions
brew services start postgresql@16 && brew services start redis
ollama pull llama3.2:3b                                        # model used by default
createdb ratemyresume                                          # any name; set POSTGRES_DB to match
```

Keycloak: download the server from keycloak.org, unzip it, and note the folder path (this becomes `KEYCLOAK_HOME`). `start.sh` expects a PostgreSQL database named `keycloak` and user `keycloak_user`:

```
psql postgres -c "CREATE USER keycloak_user WITH PASSWORD 'YOUR_KEYCLOAK_DB_PASSWORD';"
psql postgres -c "CREATE DATABASE keycloak OWNER keycloak_user;"
```

4.2 Keycloak realm (manual) [Not verified]

The code expects realm `ratemyresume`, an API audience `ratemyresume-api` in access tokens, roles `admin` and `user`, a public browser client (its id goes in NEXT_PUBLIC_KEYCLOAK_CLIENT_ID, redirect URI http://localhost:3000/*, web origin http://localhost:3000), and a service-account client `ratemyresume-admin-api` whose secret goes in KEYCLOAK_ADMIN_CLIENT_SECRET. Exact role-mapper settings are not in the repo; check `auth/security.py` for the claim paths it reads.

4.3 Python and frontend dependencies

```
cd RateMyResume/backend
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt            # includes PaddleOCR; the first install is large
cd ../frontend && npm install
```

4.4 Configure, migrate, start

Create `RateMyResume/.env` (project root, see chapter 5), then:

```
cd RateMyResume/backend && source .venv/bin/activate
alembic upgrade head          # creates the 4 tables; success: 'Running upgrade ... create core tables'
```

No seed data is required **[Not verified]**. Users are created in Keycloak, or by an admin through POST /api/v1/admin/users.

5. Environment variables and configuration

The backend reads `RateMyResume/.env` (project root, resolved in config.py). Names are case-insensitive. Unknown keys are ignored. Variables with no default are required; the app fails at startup with a validation error if they are missing. Frontend variables starting with `NEXT_PUBLIC_` are bundled into browser code and are **public**; never put secrets in them. The frontend env file location is **[Not verified]** (conventionally frontend/.env.local).

Variable	Purpose	Required?	Placeholder	Used in
APP_ENV	development or production (hides /docs in production)	No (development)	development	main.py
CORS_ORIGINS	Allowed browser origins (list)	No	["http://localhost:3000"]	main.py
POSTGRES_HOST / PORT	DB location	No	localhost / 5432	config.py
POSTGRES_DB, POSTGRES_USER	DB name and user	**Yes**	ratemyresume	config.py

Variable	Purpose	Required?	Placeholder	Used in
POSTGRES_PASSWORD	DB password	**Yes**	YOUR_DB_PASSWORD	config.py
REDIS_HOST / PORT	Redis location	No	localhost / 6379	config.py
REDIS_BROKER_DB / REDIS_CACHE_DB	Queue db / cache db	No	0 / 1	tasks, core
KEYCLOAK_URL / REALM	Identity server and realm	No	http://localhost:8080 / ratemyresume	auth
KEYCLOAK_API_AUDIENCE	Expected token audience	No	ratemyresume-api	auth
KEYCLOAK_ADMIN_CLIENT_ID / _SECRET	Service account for admin API	Needed for /admin/users	YOUR_CLIENT_SECRET	keycloak_admin.py
STORAGE_DIR	Where uploaded files are saved	No (storage/)	/path/to/storage	storage.py
MAX_UPLOAD_BYTES / MAX_BULK_FILES	Per-file limit (10 MB) / files per bulk (20)	No	10485760 / 20	resume_service
MAX_UPLOAD_REQUEST_BYTES / MAX_JSON_REQUEST_BYTES	Whole-request limits	No	219545600 / 1048576	middleware
CACHE_ENABLED, *_CACHE_TTL_SECONDS	Cache switch and lifetimes	No	true, 30, 600	cache.py
RATE_LIMIT_ENABLED	Per-user rate limiting	No	true	rate_limit.py
LLM_ENABLED	false = rubric-only, no AI text	No	true	analysis
OLLAMA_BASE_URL / MODEL / TIMEOUT_SECONDS	LLM endpoint, model, timeout	No	http://localhost:11434 / llama3.2:3b / 120	llm.py
KEYCLOAK_HOME, KEYCLOAK_ADMIN_USER, KEYCLOAK_ADMIN_PASSWORD, KEYCLOAK_DB_PASSWORD	Only for start.sh	For Keycloak	YOUR_..._PASSWORD	start.sh
NEXT_PUBLIC_API_URL	Backend base URL	No (http://localhost:8000/api/v1)	same	lib/api.ts
NEXT_PUBLIC_KEYCLOAK_URL / _REALM / _CLIENT_ID	Browser login config	**Yes** (frontend)	http://localhost:8080 / ratemyresume / YOUR_CLIENT_ID	lib/keycloak.ts

Gap: there is no `.env.example` in the repository. Development vs production: set APP_ENV=production, use a managed secret store, and never use Keycloak `start-dev`.

6. Starting, stopping, restarting

Start order: PostgreSQL and Redis, then Keycloak and Ollama, then backend API, worker, frontend. Use one Terminal window per long-running service and keep it open. The uvicorn and celery commands are not stored in the repo, so they are derived from the module paths **[Not verified]**.

Service	Directory	Start	Healthy when	Stop
PostgreSQL, Redis	any	brew services start postgresql@16 / redis	pg_isready ; redis-cli ping returns PONG	brew services stop ...
Keycloak	RateMyResume	bash infra/keycloak/start.sh	log says 'started'; http://localhost:8080 loads	Ctrl+C
Ollama	any	ollama serve (or the app)	curl localhost:11434 says 'Ollama is running'	Ctrl+C
Backend API	backend	source .venv/bin/activate; uvicorn app.main:app --reload --port 8000	curl localhost:8000/health returns status ok	Ctrl+C
Worker	backend	celery -A app.tasks.celery_app worker -l info	log shows 'ready' and both tasks listed	Ctrl+C (warm shutdown)
Frontend	frontend	npm run dev	http://localhost:3000 redirects to Keycloak login	Ctrl+C

Restart rules: changed .env or Python dependencies - restart API and worker. Changed frontend env - restart `npm run dev`. Code changes reload the API (--reload) and frontend automatically, but **not** the worker.

```
lsof -nP -iTCP:8000 -sTCP:LISTEN        # who is using a port
pgrep -fl "celery"                      # duplicate workers? keep only one
kill <PID>                              # stop a stray process (try this before kill -9)
```

Fresh-terminal checklist: 1) services running (postgres, redis, ollama) 2) Keycloak terminal 3) backend terminal: activate venv, uvicorn 4) worker terminal: activate venv, celery 5) frontend terminal: npm run dev 6) open localhost:3000, log in, upload a PDF, watch status change.

7. Frontend explained

Startup: `src/app/layout.tsx` wraps every page in `AuthProvider`, which creates a Keycloak object and calls init with onLoad 'login-required', so unauthenticated users are redirected to Keycloak. **Routes** follow the folder names under src/app: / , /dashboard, /upload, /resumes, /resumes/[id], /admin/users. **Components:** NavBar, ScoreRing (score gauge), StatusBadge, shared ui.tsx. **State:** React hooks only (useState/useEffect, a useAuth context); no state library. **Types:** src/types/api.ts mirrors backend schemas.

Traced action - viewing a resume: the page /resumes/[id] calls a function in `lib/resumes.ts`, which calls `apiFetch`. apiFetch runs `keycloak.updateToken(30)` (refresh, or log in again), sets the header `Authorization: Bearer TOKEN`, and fetches GET /api/v1/resumes/{id}. The backend validates the token, loads the owner's resume, and returns ResumeDetail JSON, which the page renders (score ring, summary, status badge). Exact loading/error UI per page: **[Not verified]** (not inspected line by line).

8. Backend and API explained

All routes below except /health require `Authorization: Bearer TOKEN`. Routes are in `backend/app/api/v1`; schemas are in `app/schemas`. Validation errors are returned by FastAPI/Pydantic; services raise typed errors mapped to HTTP codes.

Method	Endpoint	Purpose	Auth	Response
GET	/health	Postgres + Redis check	None	200 or 503, {status, checks}
GET	/api/v1/me	Current user profile	Any user	MeResponse
GET	/api/v1/admin/ping	Admin role check	admin	simple JSON
GET	/api/v1/admin/users	List Keycloak users	admin	AdminUserListResponse
POST	/api/v1/admin/users	Create user	admin	201 AdminUserOut
GET	/api/v1/dashboard/stats	Aggregate stats (cached)	user	DashboardStats
POST	/api/v1/resumes	Upload one file (multipart)	user	202 UploadItemResult
POST	/api/v1/resumes/bulk	Upload up to 20 files	user	202 BulkUploadResponse
GET	/api/v1/resumes	List own resumes (filter/paginate)	user	ResumeListResponse
GET	/api/v1/resumes/{id}	Resume detail	user (owner)	ResumeDetail
GET	/api/v1/resumes/{id}/status	Processing status	user (owner)	ResumeStatus
POST	/api/v1/resumes/{id}/retry	Re-queue failed work	user (owner)	202 ResumeStatus
DELETE	/api/v1/resumes/{id}	Delete resume	user (owner)	204
POST	/api/v1/resumes/{id}/analyze	Run/re-run analysis	user (owner)	202 ResumeStatus
GET	/api/v1/resumes/{id}/analysis	Latest analysis (cached)	user (owner)	AnalysisResponse

Endpoints were read from source, not executed, so none is marked 'tested'. Examples (placeholders; the multipart field name must be confirmed in `api/v1/resume.py` **[Not verified]**):

```
curl http://localhost:8000/health
curl -H "Authorization: Bearer $TOKEN" http://localhost:8000/api/v1/me
curl -H "Authorization: Bearer $TOKEN" -F "file=@resume.pdf" http://localhost:8000/api/v1/resumes
curl -H "Authorization: Bearer $TOKEN" http://localhost:8000/api/v1/resumes/RESUME_ID/status
```

Get a token by logging in through the browser and copying it from the app, or via a Keycloak token request; `backend/scripts/verify_*.py` show how the project does it. With APP_ENV=development, interactive docs are at /docs.

9. Database explained

PostgreSQL via SQLAlchemy (driver `postgresql+psycopg`); the URL is built in `core/config.py`, sessions in `db/session.py`. Tables and relationships are shown in Figure 3. Key rules: unique (owner_id, file_hash) blocks duplicate uploads; check constraints enforce valid statuses, positive file size, 64-char hash, score 0-100; indexes on (owner_id, uploaded_at), (owner_id, extraction_status), (resume_id, job_type), (resume_id, created_at). JSONB columns store skills and the full analysis result.

Migrations are versioned scripts in `backend/alembic/versions`. There is one (core tables).

```
cd backend && source .venv/bin/activate
alembic current                 # which version is applied
alembic upgrade head            # apply new migrations
alembic history                 # list migrations
psql -d ratemyresume -c "\dt"   # list tables
```

Safe inspection queries (read-only):

```
SELECT id, original_filename, extraction_status, analysis_status, uploaded_at
  FROM resumes ORDER BY uploaded_at DESC LIMIT 10;
SELECT resume_id, job_type, status, attempt_count, last_error
  FROM processing_jobs WHERE status IN ('queued','failed') ORDER BY started_at NULLS FIRST;
SELECT resume_id, score, model_name, created_at
  FROM resume_analyses ORDER BY created_at DESC LIMIT 10;
```

Connection refused? Check `pg_isready`, then POSTGRES_HOST/PORT/USER/PASSWORD in the root .env.

10. Resume ingestion, extraction, OCR, and AI analysis

1 Validate: the file type is detected from its first bytes (%PDF- or DOCX zip), not from the file name; only PDF and DOCX are accepted; max 10 MB each; SHA-256 hash blocks duplicates per user. **2 Store:** saved under STORAGE_DIR as owner-id/random-uuid.ext (original name kept only in the database). **3 Queue:** a resume row and queued job are committed, then the task is enqueued. **4 Extract:** text is read directly from the file; if a PDF has little or no text it is a *scanned* PDF, so pages are rendered and read by PaddleOCR. Text is capped at 200,000 characters. **5 Identify:** email and phone by regex; name, location, skills by the LLM (skills must literally appear in the text). **6 Score:** deterministic rubric (contact 20, structure 30, skills 15, impact 20, length/dates 15). **7 Review:** the LLM writes summaries, strengths, and improvements; output is cleaned and validated. **8 Save:** one row in resume_analyses.

Key differences. A digital PDF contains real text; a scanned PDF is only pictures. Text extraction reads existing characters; OCR guesses characters from images (slower, less exact). A prompt is the instruction and data sent to the LLM; the response is its answer. Synchronous work finishes inside the request; background work finishes later. A **failure** means the pipeline could not complete (status failed); a **low score** is a successful analysis of a weak resume.

Where to look when it fails: resumes.error_message (safe text), processing_jobs.last_error (internal), worker terminal log, services/processing.py, ai/ocr.py, and Ollama reachability. A real-file walkthrough was not run **[Not verified]**; to do one, upload a PDF and watch GET .../status while tailing the worker log.

11. Authentication, authorization, and security

Identity provider vs backend: Keycloak proves who the user is and issues a JWT (a signed token with claims such as user id and roles). The backend never sees passwords; it downloads Keycloak's public keys (JWKS) and verifies each token's signature (RS256), issuer, audience, and expiry. **Authentication** = who you are; **authorization** = what you may do (roles admin and user via `require_role`). A local user_profiles row is created and linked by keycloak_sub. Resume queries are scoped by owner_id; whether tests cover cross-user access **[Not verified]**.

Control	Status
JWT validation, role checks	Present (auth/security.py)
Upload validation: magic bytes, size limits, hash dedupe, random storage names, path-traversal guard	Present
Rate limiting (Redis), request-size limits, security headers, CORS allow-list, /docs off in production	Present
Prompt-injection hardening of the LLM prompt; safe user-facing error messages; internal errors kept in last_error/logs	Present
Secrets via .env, SecretStr in config; .env excluded from the repo	Present; no .env.example
HTTPS / reverse proxy, virus scanning, encryption at rest, retention policy	Not present
Whether DELETE /resumes/{id} also removes the stored file	Not verified

12. Background jobs and reliability

Redis db0 is the Celery broker (queue). Postgres is the source of truth: Celery results are ignored, and each job row records status and attempts. A worker **claims** a job with an atomic queued-to-processing update, so two workers cannot run the same job. Settings: task_acks_late (a task is acknowledged only after it finishes, so a crashed worker's task is redelivered), JSON serialization. Retry delay is min(5 x 2^attempt, 120) seconds; the cap on attempts is processing_jobs.max_attempts (default 3). Extraction chains into analysis on success.

If the database commit succeeds but enqueueing fails (e.g. Redis down): enqueue never raises; the job stays *queued*. Existing recovery: POST /resumes/{id}/retry, and scripts `backend/scripts/enqueue_queued_*.py` and `requeue_resume.py`. **Recommended (not present):** a periodic task to re-enqueue stale queued jobs and detect jobs stuck in processing, plus worker monitoring.

13. Testing and debugging

```
cd backend && source .venv/bin/activate && pytest          # tests/ (pytest listed in requirements: Not verified)
cd frontend && npm run lint                                 # ESLint; there is no frontend test script
cd frontend && npx tsc --noEmit                             # type check (tsconfig present)
python scripts/verify_security.py                           # end-to-end checks need the full stack running
curl localhost:8000/health ; pg_isready ; redis-cli ping
```

There is no backend lint/format configuration found in the repo **[Not verified]**. Logs are the terminal output of each service.

Problem	Diagnose / fix
Port in use	lsof -nP -iTCP:PORT -sTCP:LISTEN, then stop the process
Backend will not start: validation error	A required POSTGRES_* variable is missing in the root .env
DB connection refused	pg_isready; check POSTGRES_* values; brew services list
401 / invalid token	Token expired, wrong realm/audience (KEYCLOAK_*); check auth/security.py logs
403	User lacks the admin/user role in Keycloak
Browser CORS error	Add the frontend origin to CORS_ORIGINS; restart API. A 413 shows as CORS if the body is too large
Frontend cannot reach backend	NEXT_PUBLIC_API_URL, API running on :8000, restart npm run dev after env changes
Upload rejected	Only PDF/DOCX, up to 10 MB, no duplicate of an earlier upload; check the response message
Empty extracted text / OCR failure	Scanned PDF? Check PaddleOCR install and ai/ocr.py; run backend/smoke_test_ocr.py
LLM unavailable	ollama list; curl localhost:11434; OLLAMA_MODEL pulled? Or set LLM_ENABLED=false for rubric-only
Redis down	redis-cli ping; cache and rate limits fail open, but jobs stay queued
Job stuck in queued	Is the worker running? Retry endpoint or enqueue_queued_* script
Migration mismatch	alembic current vs alembic history; alembic upgrade head

14. Making changes safely

Task	Where / steps	Verify
Add a frontend page	Create frontend/src/app/NAME/page.tsx; add a link in components/NavBar.tsx; fetch via a function in src/lib	npm run dev, npm run lint
Add an API endpoint	Add a route in api/v1/*.py (with Depends(require_role)); logic in services/; schema in schemas/; mirror type in frontend/src/types/api.ts	curl, new test in tests/
Add a database field	Edit the model in app/models; run alembic revision --autogenerate -m 'msg'; review the file; alembic upgrade head; update schemas	alembic current; psql \d table
Change scoring	services/rubric.py (bump rubric version); update tests/test_rubric.py	pytest tests/test_rubric.py
Change the LLM prompt	ai/analysis.py (bump prompt_version)	Re-run analysis on a sample resume
Add an env variable	Add a field in core/config.py with a default; use get_settings(); document it	Restart API and worker
Add a test	New tests/test_*.py file next to similar ones	pytest
Investigate a bug	Reproduce, read the worker/API log, query resumes and processing_jobs, then read the matching service	Add a regression test

15. Deployment and production considerations

Current deployment configuration: none (no Dockerfile, compose file, or CI in the repo). Keycloak runs in dev mode only. The app is not shown to be production-ready. Needed before production (recommendations): production Keycloak with HTTPS and a realm export; managed secrets; managed Postgres with backups; durable shared storage instead of a local folder; process supervision and monitoring for the API and worker; structured logging; the stale-job recovery task above; HTTPS reverse proxy; confirmed file-deletion and data-retention behavior; load-testing Ollama capacity. Already present: /health, upload and request limits, rate limiting, APP_ENV switch.

16. Glossary and quick reference

Term	Meaning
API / REST / endpoint	A way for programs to talk over HTTP; each URL+method is an endpoint
Frontend / backend	What runs in the browser / what runs on the server
Router / service layer	Code that receives requests / code that does the actual work
ORM, SQL, migration	Python objects mapped to tables; the database query language; a versioned schema change
Primary / foreign key	A row's unique id / a link to another table's row
JSON, JSONB, UUID	Text data format; its binary Postgres version; a random unique id
JWT, OIDC, RBAC	A signed login token; the login standard Keycloak uses; permissions by role
OCR, LLM, prompt	Reading text from images; a language-model AI; the instructions sent to it
Redis, Celery, worker	In-memory store; job-queue library; the process that runs jobs
Asynchronous task	Work that finishes after the request returns
Environment variable	A setting supplied outside the code
CORS	Browser rule about which websites may call an API
Retry / idempotency	Trying again after failure / repeating an action safely without duplicate effects
Health check	An endpoint that reports whether the service works (/health)

Service and port reference (from config defaults)

Service	Port
Frontend (Next.js)	3000 (Next.js default)
Backend API	8000 (assumed by frontend default URL)
PostgreSQL	5432
Redis	6379
Keycloak	8080
Ollama	11434

Where should I look?

Task or error	File
Login or token problems	backend/app/auth/security.py, frontend/src/lib/keycloak.ts, api.ts
Upload rejected	backend/app/services/resume_service.py
Wrong score	backend/app/services/rubric.py
Bad AI text	backend/app/ai/analysis.py
Job stuck or retrying	backend/app/services/processing.py, app/tasks/
Setting missing	backend/app/core/config.py
Table change	backend/app/models, alembic/versions

Outstanding gaps / next steps: add a root README and .env.example; add Docker compose for Postgres, Redis, Keycloak; export the Keycloak realm; add frontend tests and CI; add the stale-job recovery task; document the upload field name and worker command in-repo; verify the items tagged **[Not verified]**.